

Travaux Pratiques

Module M214 - Application Cloud Native

Série d'Exercices : Structure & Organisation

🎯 Objectifs

- Organiser un projet Express avec une architecture professionnelle
- Séparer les responsabilités : Routes, Controllers, Models
- Créer des modules utilitaires réutilisables
- Appliquer les bonnes pratiques de développement

📁 Exercice 1 : Structure de Projet

Jusqu'à présent, tout le code était dans un seul fichier. Dans un projet réel, on organise le code en plusieurs dossiers et fichiers.

Structure cible :

```
1 api-livres/  
2 |-- models/  
3 |   |-- Livre.js  
4 |-- routes/  
5 |   |-- livreRoutes.js  
6 |-- index.js  
7 |-- package.json
```

Schéma Livre : titre (String, required), auteur (String, required), prix (Number, min : 0), pages (Number), genre (String), disponible (Boolean, default : true)

- Q1.** Créez le fichier `models/Livre.js` avec le schéma complet et l'export du modèle.
- Q2.** Créez le fichier `routes/livreRoutes.js` qui contient toutes les routes CRUD utilisant `express.Router()`. N'oubliez pas d'importer le modèle et d'exporter le router.
- Q3.** Créez le fichier `index.js` qui : importe `express` et `mongoose`, configure le middleware JSON, connecte à MongoDB, importe et utilise les routes avec `app.use('/api/livres', livreRoutes)`, et démarre le serveur.
- Q4.** Quel est l'avantage de séparer les routes dans un fichier distinct ?
- Q5.** Si on veut ajouter une entité "Auteur", quels fichiers doit-on créer ?

⚙️ Exercice 2 : Controllers

Les **controllers** contiennent la logique métier. Les routes ne font qu'appeler les fonctions du controller.

Structure cible :

```
1 api-livres/  
2 |-- models/  
3 |   |-- Livre.js  
4 |-- controllers/  
5 |   |-- livreController.js  
6 |-- routes/  
7 |   |-- livreRoutes.js  
8 |-- index.js
```

Exemple de controller :

```
1 // controllers/livreController.js  
2 const Livre = require('../models/Livre');  
3  
4 exports.getAllLivres = async (req, res) => {  
5   try {  
6     const livres = await Livre.find();  
7     res.json(livres);  
8   } catch (error) {  
9     res.status(500).json({ error: error.message });  
10  }  
11 };
```

Exemple de route utilisant le controller :

```
1 // routes/livreRoutes.js  
2 const express = require('express');  
3 const router = express.Router();  
4 const livreController = require('../controllers/  
   livreController');  
5  
6 router.get('/', livreController.getAllLivres);  
7 module.exports = router;
```

Q1. Créez `controllers/livreController.js` avec les 5 fonctions CRUD : `getAllLivres`, `getLivreById`, `createLivre`, `updateLivre`, `deleteLivre`.

Q2. Modifiez `routes/livreRoutes.js` pour utiliser les fonctions du controller au lieu d'avoir la logique directement dans les routes.

Q3. Ajoutez une fonction `getLivresByGenre` dans le controller qui filtre les livres par genre passé en paramètre.

Q4. Ajoutez une fonction `getLivresDisponibles` qui retourne uniquement les livres avec `disponible: true`.

Q5. Quelle est la différence entre `exports.maFonction` et `module.exports = { maFonction }`?

Q6. Pourquoi séparer la logique dans des controllers au lieu de la mettre dans les routes?

✂ Exercice 3 : Modules Utilitaires

Les **modules utilitaires** (utils/helpers) contiennent des fonctions réutilisables dans tout le projet.

Structure cible :

```
1 api-livres/  
2 |-- models/  
3 |-- controllers/  
4 |-- routes/  
5 |-- utils/  
6 |     |-- responseHelper.js  
7 |     |-- validators.js  
8 |-- config/  
9 |     |-- db.js  
10 |-- index.js
```

Q1. Créez `config/db.js` qui exporte une fonction `connectDB()` pour la connexion MongoDB. Utilisez cette fonction dans `index.js`.

Q2. Créez `utils/responseHelper.js` qui exporte les fonctions suivantes :

- `sendSuccess(res, data, statusCode = 200)` - envoie `{success: true, data}`
- `sendError(res, message, statusCode = 500)` - envoie `{success: false, error: message}`
- `sendNotFound(res, resource = 'Ressource')` - envoie une erreur 404

Q3. Refactorisez le controller pour utiliser les fonctions de `responseHelper.js` au lieu de `res.json()` et `res.status()`.

Q4. Créez `utils/validators.js` avec une fonction `isValidObjectId(id)` qui vérifie si un ID est un `ObjectId` MongoDB valide (utilisez `mongoose.Types.ObjectId.isValid()`).

Q5. Utilisez `isValidObjectId` dans le controller pour valider l'ID avant d'appeler `findById`. Retournez une erreur 400 si l'ID est invalide.

Q6. Créez une fonction `formatLivreResponse(livre)` dans un fichier `utils/formatters.js` qui retourne un objet avec uniquement : `id`, `titre`, `auteur`, `prix` (sans `__v`, `createdAt`, `updatedAt`).

🔧 Exercice 4 : Projet Complet - API Gestion de Films

Créez une API complète pour gérer des films en appliquant toutes les bonnes pratiques vues.

Structure finale :

```
1 api-films/  
2 |-- config/  
3 |   |-- db.js  
4 |-- models/  
5 |   |-- Film.js  
6 |-- controllers/  
7 |   |-- filmController.js  
8 |-- routes/  
9 |   |-- filmRoutes.js  
10 |-- utils/  
11 |   |-- responseHelper.js  
12 |   |-- validators.js  
13 |-- index.js
```

Schéma Film : titre (required), realisateur (required), annee (Number, min : 1895, max : année courante), duree (Number, en minutes), genre (enum : Action, Comedie, Drame, Horreur, ScienceFiction, Animation), note (Number, min : 0, max : 10), synopsis (String), affiche (Boolean, default : false)

- Q1. Créez la structure complète du projet avec tous les fichiers.
- Q2. Implémentez le modèle Film avec toutes les validations.
- Q3. Implémentez le controller avec les fonctions : `getAllFilms`, `getFilmById`, `createFilm`, `updateFilm`, `deleteFilm`, `getFilmsByGenre`, `getFilmsByYear`, `getTopRatedFilms` (note ≥ 8).
- Q4. Implémentez les routes correspondantes.
- Q5. Ajoutez une route `GET /api/films/search?titre=xxx` qui recherche les films dont le titre contient le terme recherché (utilisez `$regex`).
- Q6. Ajoutez une route `GET /api/films/stats` qui retourne : le nombre total de films, la note moyenne, le film le plus ancien, le film le mieux noté.
- Q7. Dessinez un schéma montrant le flux d'une requête : Client Route Controller Model MongoDB Response.